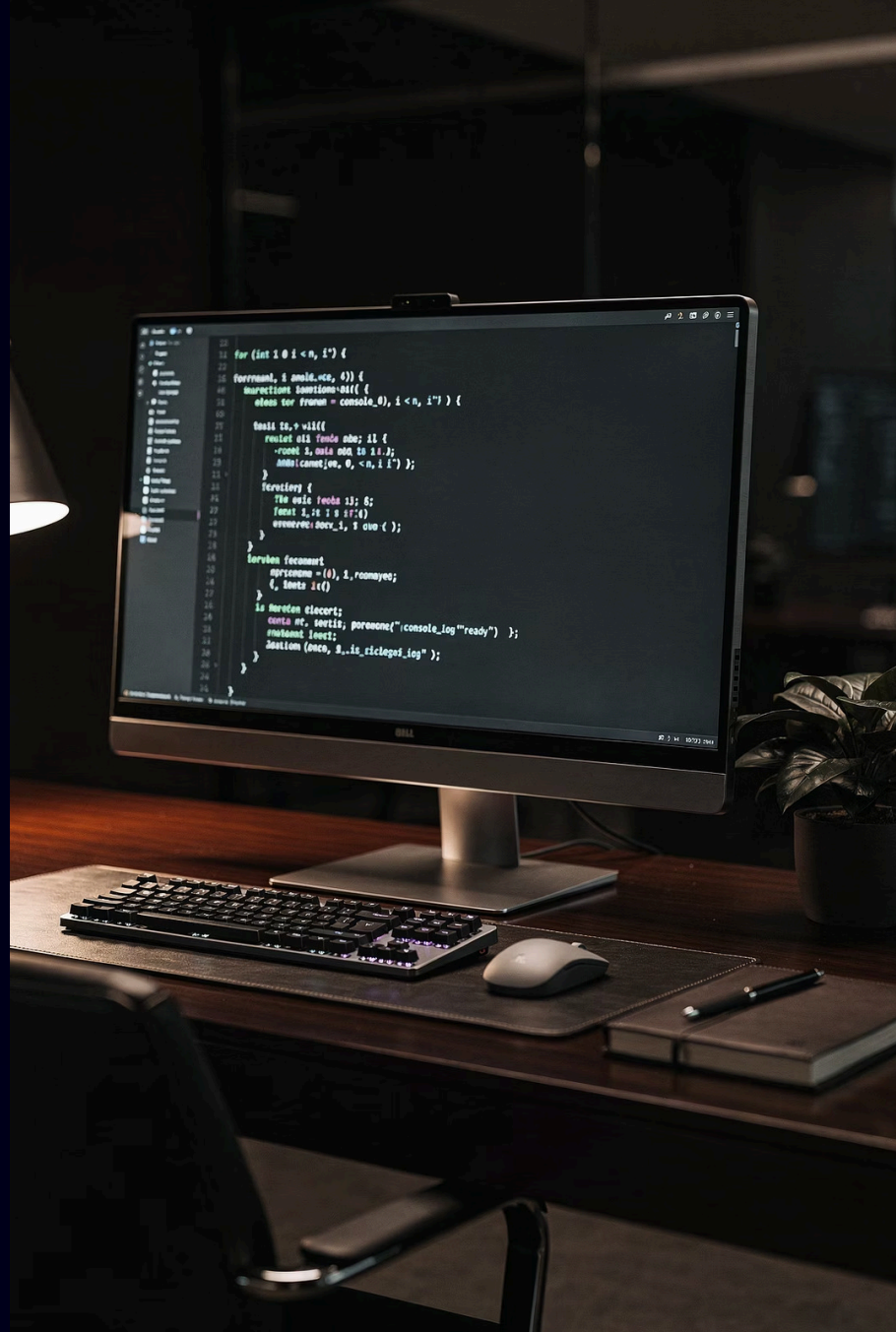
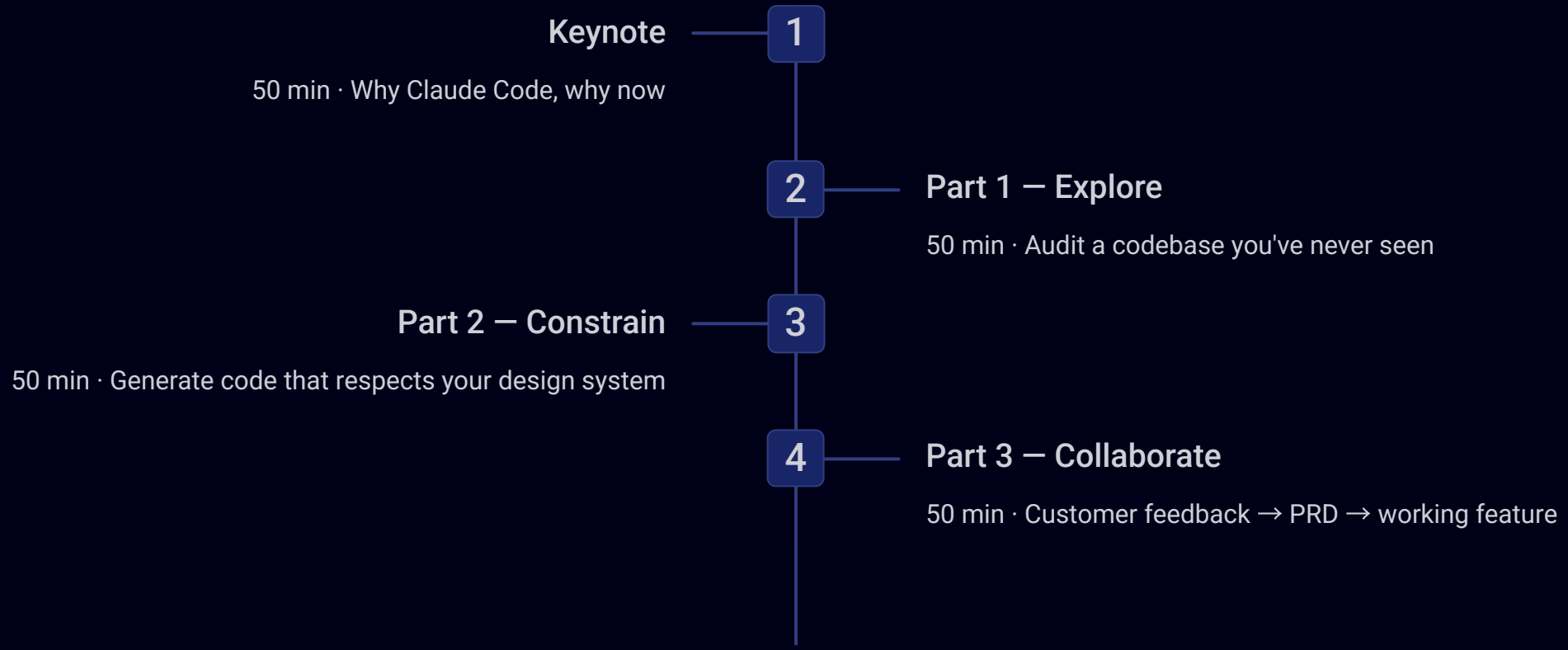
Claude Code for Product Teams

4 hours · 3 skills · One Thursday

Shmulik Davar · BrAIght Wave



Three hours from now, you'll have an AI co-worker.



Then a Friday commitment. Keynote is your map. Parts 1–3 are hands-on with Claude Code on your laptop.

Round the table.

Within your pod: 60 seconds per person, max. *All 10 pods at the same time — you'll be done in 4 minutes.*

1. Name
2. Role
3. A product you ship
4. One thing you want from today

- [ ] 📝 Write your "one thing" on a sticky note. Post it on the table. We'll come back to it.

Shmulik Davar

Founder and CEO of BrAIght Wave, helping organizations design, build, and scale production-grade AI and GenAI products. I work at the intersection of strategy, product, and deep AI execution — turning emerging technology into measurable business impact.

Building Production AI Systems That Matter

I specialize in taking AI from idea to reality. From defining product vision and AI strategy to shipping and scaling real-world systems, my focus is on outcomes, adoption, and long-term value.

At BrAIght Wave, I advise executives and product teams on how to responsibly and effectively embed AI into their core products, operations, and decision-making.

Areas of Expertise

- **AI & GenAI Product Strategy** — End-to-end ownership of AI and GenAI products: discovery, roadmap, architecture, go-to-market.
- **Enterprise AI & Advisory** — Supporting leadership teams on AI adoption, operating models, governance, and execution at scale.
- **ML Platforms & Enablement** — Designing internal platforms, tooling, and processes that enable ML and data teams to deliver faster.

Experience: BrAIght Wave · FIDO · Spark Beyond · MEDINT · Bringg · parola · Seconds

THE SHIFT

It's not a feature. It's the platform.

Every digital product is being rewired with intelligence, automation, and natural language interfaces. How we build them — and how PMs manage them — looks fundamentally different now. This isn't an upgrade cycle. It's a complete architectural reset of what software is and who can build it.

PRODUCT LEADERSHIP | 2026

The Modern PM & Designer

What's actually expected of you in 2026.

Five expectations

For PMs and designers who want to stay relevant, the job is getting broader, faster, and more collaborative.

🚀 Ship faster

Your VP doesn't want a 6-week PRD anymore. Speed is the new baseline.

🔬 Synthesize more

200 customer comments, 4 stakeholder calls, last quarter's data — all by Tuesday.

💻 Translate between code and product

Your engineers expect you to know what's in the codebase before you scope anything.

📅 Onboard to new products in days

Not the 90-day "I'm new here" excuse. Days. With the right tools, that's real.

🤖 Work with AI as a teammate

Not as a chat window you copy-paste in and out of. The PMs who make this shift ship more, scope better, and onboard faster. **That's what tonight is about.**

FOMA

FOMA

Fear Of Missing AI **...and why it's the wrong frame.**

The instinct to panic-adopt every new AI tool is understandable — but it's a trap. FOMA leads to shallow experimentation, scattered tooling, and no real compounding advantage. The PMs winning in 2026 aren't the ones who used the most tools. They're the ones who understood the shift deeply enough to make better decisions faster.

Depth beats breadth. Every time.

Global Trends in the World of AI

Four seismic forces are reshaping the AI landscape — and every product team needs to understand where each one is heading.

Automation & Efficiency

AI continues to improve operational efficiency, cutting costs and increasing productivity across every business function — from support to engineering.

Generative & Agentic AI

AI now generates content, code, and ideas — and increasingly executes multi-step tasks autonomously, without human hand-holding at each step.

Accessibility & Democratization

Vibe coding, no-code AI, and natural language interfaces let non-technical users build real products. The bar to ship has collapsed.

Responsible AI & Governance

EU AI Act, evals, observability, and MCP security are now production requirements — not nice-to-haves. Compliance is a PM responsibility.

What the workspace actually looks like in 2026.

Connected agents replace the 12 tabs you used to switch between. A single AI agent — powered by MCP — sits at the center of your entire tool stack.

Connected through **MCP — the Model Context Protocol.** Instead of switching tabs, your AI agent reads, writes, and acts across your entire stack — creating tickets, scheduling meetings, summarizing threads, and shipping code — all from one interface.

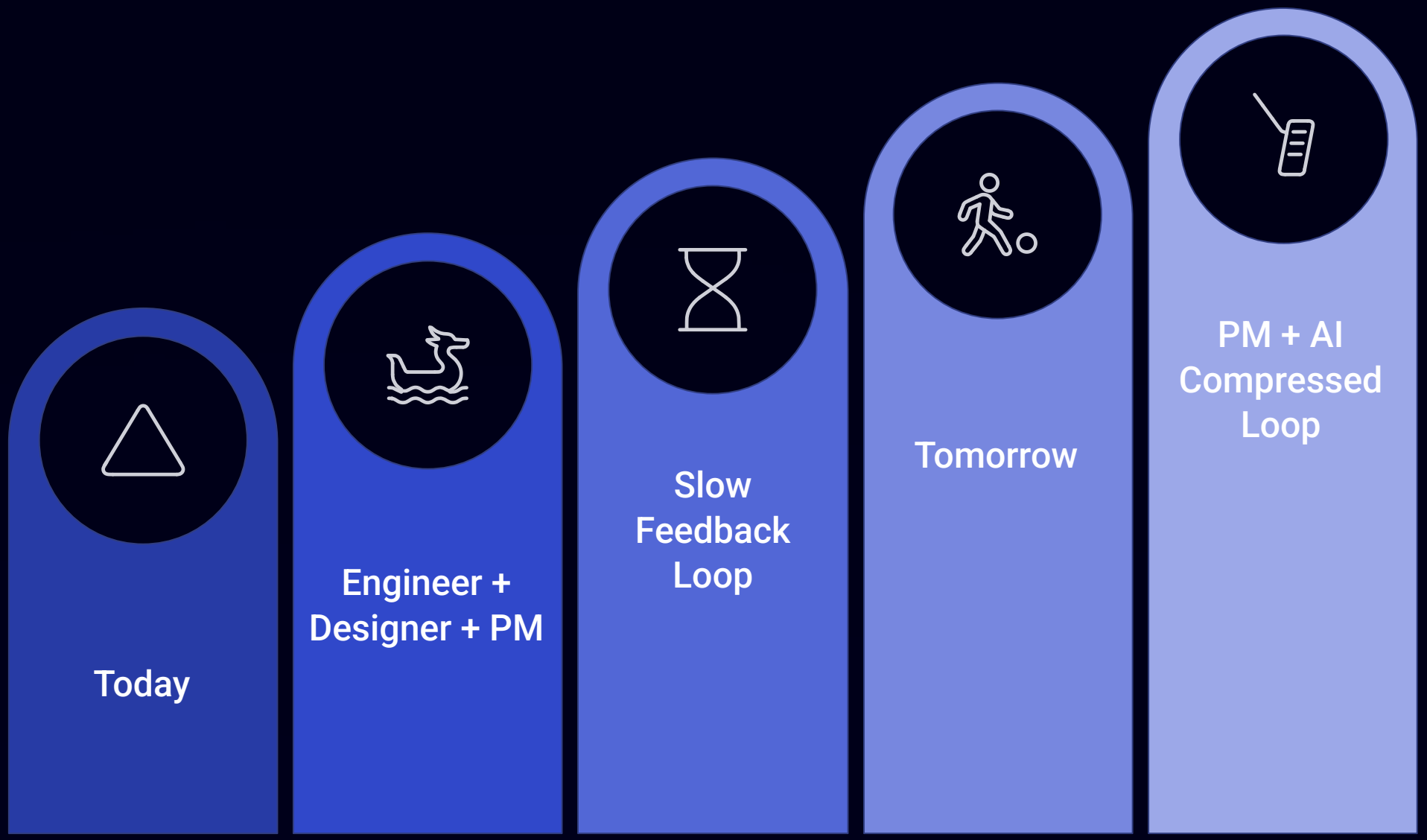
The product team is being rewired.

Today

Separate specialists: **Engineer + Designer + PM** each own distinct lanes. The PM coordinates, but rarely creates artifacts directly. The feedback loop is slow — idea → spec → design → build → test → ship.

Tomorrow

The **PM + AI** compresses the entire loop. PM ↔️ artifact ↔️ test ↔️ ship. One person with the right tools and taste can do in an afternoon what used to take a two-week sprint.



📝 The PM is no longer the slowest hand on the keyboard. AI eliminates the translation layer between idea and execution. What changes is not the PM's role — it's the **speed and leverage** of that role.

CAREER PATHS

Four Paths for AI Product Managers

AI Platform PM

Tools and infrastructure for AI engineers and data scientists.

Focus: Technical depth, model performance, developer experience.

AI Product PM

AI-based solutions that solve real-world business problems.

Focus: Use case definition, model selection, performance metrics.

AI-Powered PM

Leveraging AI to optimize traditional PM processes end-to-end.

Focus: PM productivity, insight generation, decision support.

🆕 AI Agent PM

Designing autonomous systems that plan, use tools, and act.

Focus: Agent reliability, evals, tool design, human-in-the-loop.

Skills of an AI-Powered PM

Master these to excel as a PM in the AI era. The best PMs in 2026 don't just use these tools — they develop fluency across all of them.

Multiple Chatbots

Know the best tool for each task — don't default to one model for everything.

Chat as Thought Partner

AI copilot mindset — use it for brainstorming, stress-testing, sense-checking.

Automates Day-to-Day

GPTs, Zapier, n8n, Tasks — automate the repetitive work that steals your focus.

Deep Research

Perplexity, ChatGPT Deep Research — synthesize markets and competitors in minutes.

Multimodal

Image + video analysis — extract insight from visual data, not just text.

Voice Dictation

Wispr Flow, Whisper — capture ideas at the speed of thought, hands-free.

Meeting Summarization

Granola, TimeOS, Fireflies — never manually write meeting notes again.

Prompt Improvement

Prompt generators + libraries — craft better instructions, get better outputs.

Data Analysis

ChatGPT, NotebookLM, Claude — turn raw data into actionable insights.

Vibe Prototyping

Bolt, Lovable, v0, Base44 — ship working prototypes before writing a spec.

AI Mindset

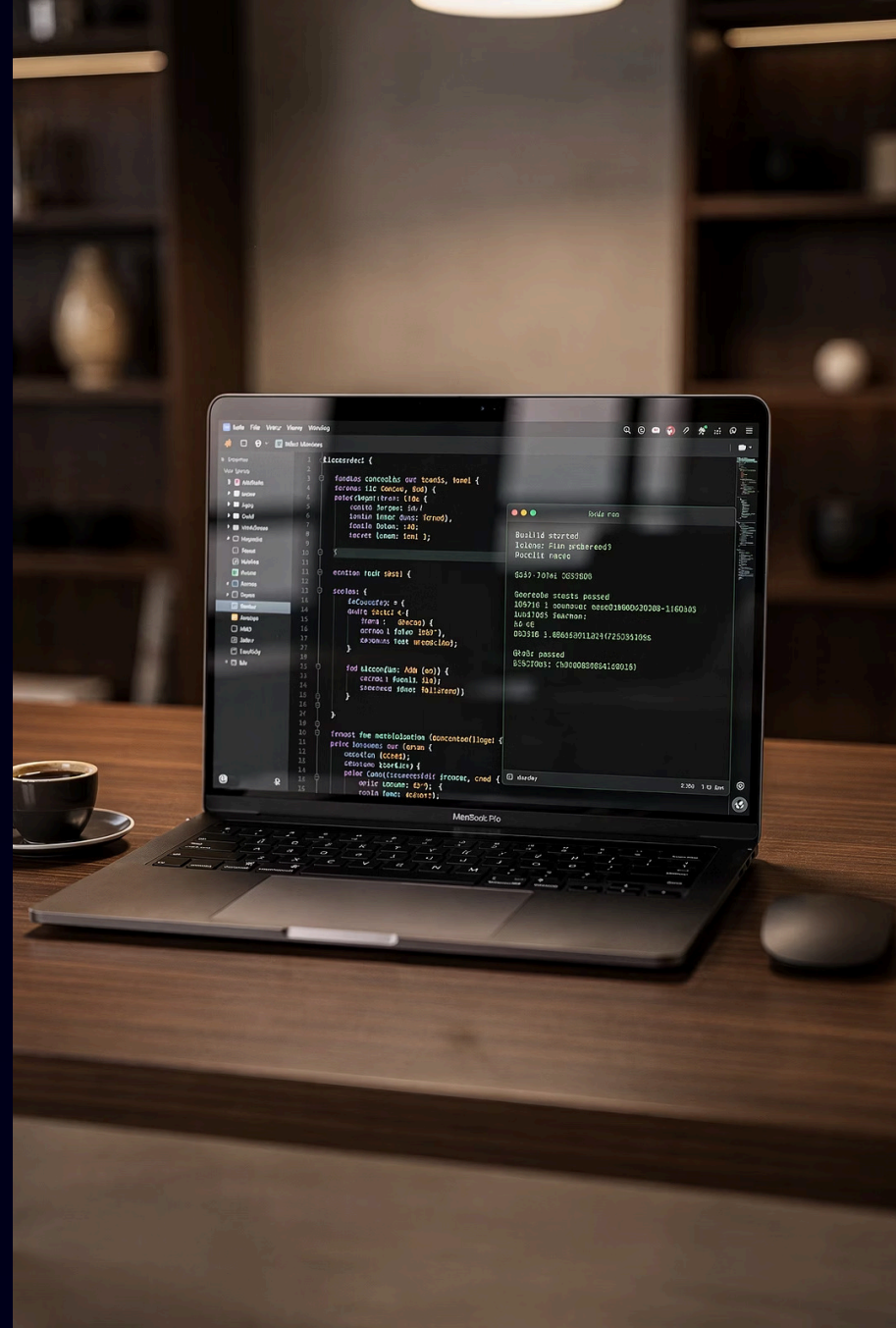
Spots use cases everywhere — always asking "where can AI create leverage here?"

Continuous Refinement

Treat every output as a starting point — iterate, evaluate, improve constantly.

Meet Claude Code

Now — let's actually meet the tool we're using tonight.



The Claude Family

Four products. One company. Different jobs.

Product	What it is	Use it for
Claude.ai (browser)	The chat window most people know.	Quick thinking, drafting, research. No file access.
Claude Desktop — Cowork mode	An agent for daily knowledge work.	Research synthesis. Document creation. Scheduled tasks. No coding required.
Claude Code ⭐	An agent for code-grounded work.	Reads your real files. Scoping, PRDs, feature audits, design-system enforcement. **Tonight.**
Claude API	For developers building on top.	Not us today.

ⓘ The big distinction: **Cowork for knowledge work that doesn't touch a codebase. Claude Code for work that does.** Same app. Different power.

WHY CLAUDE CODE TONIGHT

Three reasons we go deep on Claude Code tonight.

Agentic at long horizons

Stays coherent over 30-minute multi-file tasks. The patterns hold up where alternatives degrade. You'll feel this in Part 1.

Plan mode is safe collaboration

Read-only exploration. Approve in one keystroke. You can't break what you're not editing. This is the safety story for non-coders.

Skills + CLAUDE.md = a team library

What you save tonight, your teammate runs next week. No equivalent in Cursor, Codex, or Copilot.

Other tools are catching up. The patterns matter more than which logo wins. Plan mode, skills, CLAUDE.md — these transfer.

Cowork in 5 minutes.

What you're about to watch: Cowork reads a folder of 200 customer feedback rows and produces a 1-page synthesis — top themes, sentiment, representative quotes. **No copy-paste. No "format this for me." Just done.**

🔍 Research synthesis

Drop a folder. Get a structured report with top themes, sentiment, and representative quotes.

📄 Document creation

From raw notes to polished output — no manual formatting required.

📅 Scheduled tasks

Set it and come back to results. Cowork works while you focus elsewhere.

Demo prompt: "Synthesize this customer feedback into a 1-page report — top 3 themes by impact, with representative quotes." Drop the workshop-data/ folder containing feedback.csv.

DECISION FRAMEWORK

A simple decision tree.

1

"Help me think / draft this email"

→ **Claude.ai in the browser.** Fast. Zero overhead. No files needed.

2

"I have a folder of feedback / notes"

→ **Cowork.** Drop the folder. Get the synthesis. No code required.

3

"I need to look at the actual codebase"

→ **Claude Code ⭐ Tonight.** Reads real files. Grounds every answer in evidence.

📝 **If you remember nothing else from Part 1:** Cowork for knowledge work. Claude Code for code-grounded work. Both live inside the same Claude Desktop app.

HONEST ABOUT SCOPE

What this workshop is NOT.

AI ethics, bias & hallucination management

Real. Important. Not 4-hour content. We'll point you at the right resources in the closing.

AI evals & observability for production features

A specialty in itself. Aman Khan's deeplearning.ai course is the canonical resource.

Multi-tool coverage

No ChatGPT + Gemini + Cursor + n8n comparison tour. We go deep on one. The patterns transfer.

These are real, important topics. None of them get justice in 4 hours alongside everything else. The closing slide tells you where to go for the rest.

THE WORKSHOP FRAMEWORK

Same Claude Code. Three different relationships.

🔎 Part 1 — Explore

Claude as your research partner.

You don't know the codebase. Claude does. Ask, get evidence, scope the work.

🎯 Part 2 — Constrain

Claude as your craftsperson, with rules.

Generate code that follows your design system — not its imagination.

🤝 Part 3 — Collaborate

Claude as both your PM-self and your Dev-self.

Round-trip a feature from spec to code to revised spec — in one session.

Three relationships. Three skills you'll save. The first 3 entries of your team's Claude Code library.

Plan Mode

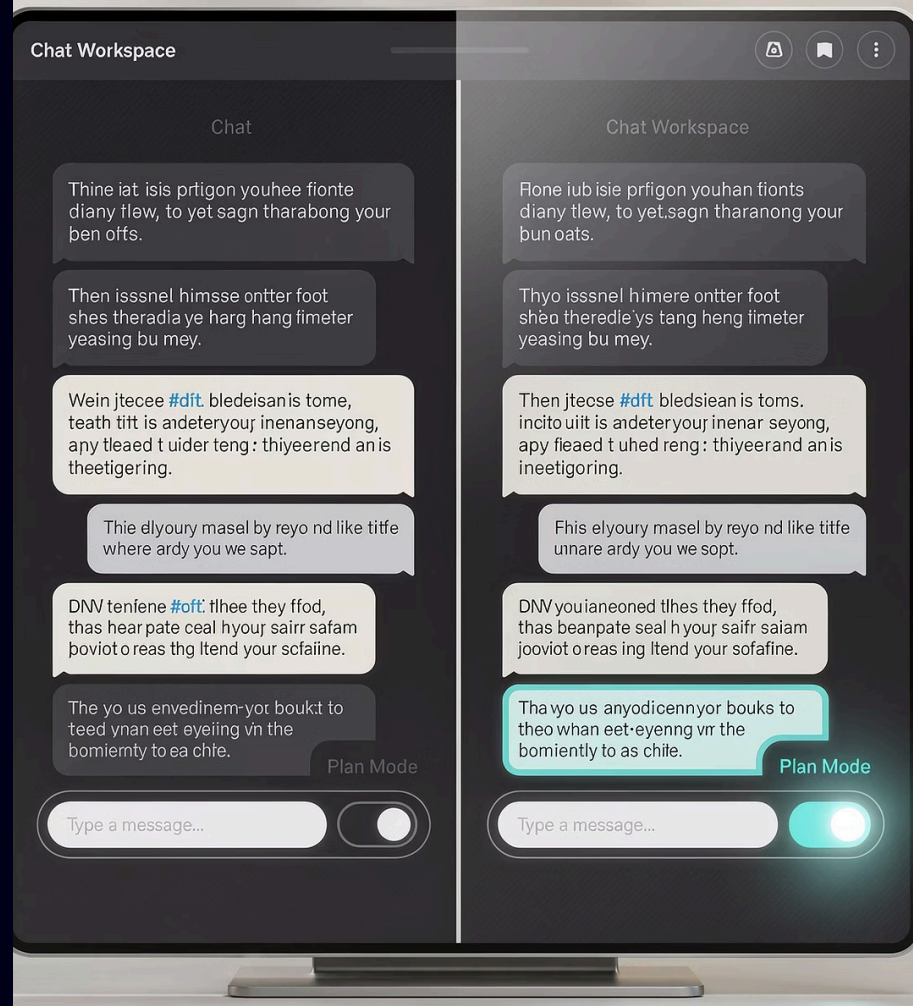
Read-only exploration. Toggle near the input.

- **On** — Claude reads your files. Cannot edit anything.
- **Off** — Claude can write, edit, run commands.

Use it for: surveys, audits, scoping, "I just want to understand this codebase."

Toggle off when: you're ready for Claude to actually change something.

> *You'll use Plan Mode in Step 1 of every exercise tonight. It's the safety story for non-coders — you literally cannot break what you're not editing.*



THE FUNDAMENTALS · 2 OF 4

📋 CLAUDE.md

The project memory file. Lives at the repo root.

This markdown file is automatically read by Claude at the start of every session. It explicitly tells Claude the product context, coding conventions, files to ignore, and key search patterns. Think of it as the ultimate onboarding document you wish every new hire had.

Tonight's repo already includes one. Claude is actively using it, so you don't have to manually remind it of project specifics.

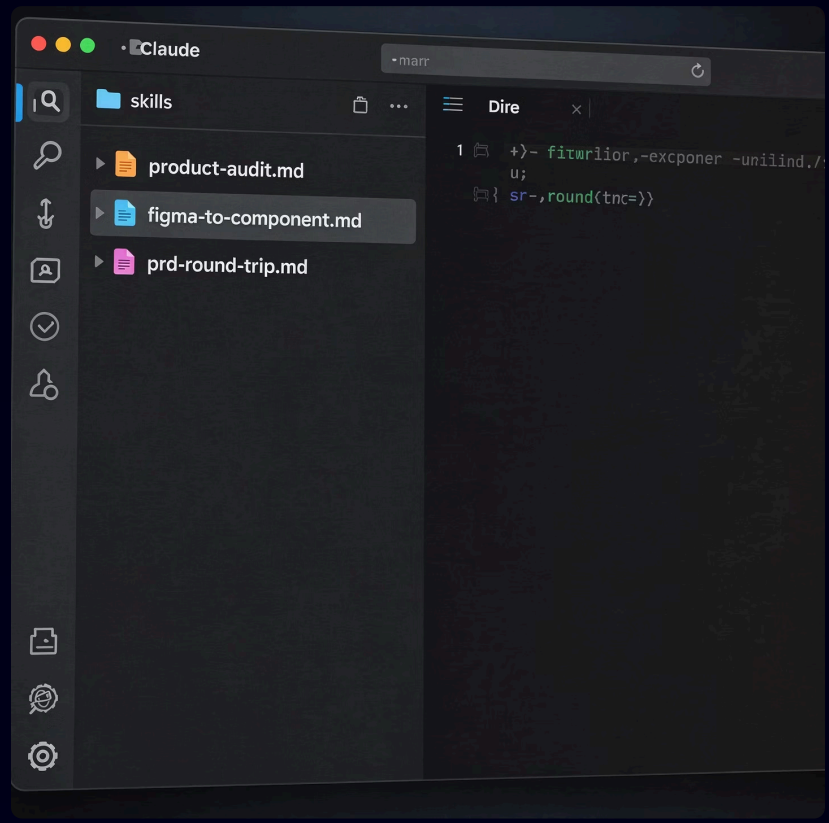
THE FUNDAMENTALS · 3 OF 4

🛠️ Skills

Reusable prompt patterns saved as Markdown.

- Live in `.claude/skills/` — one Markdown file per skill.
- Invoke with: `"Use the {skill-name} skill on this codebase."`
- Three skills tonight: `product-audit`, `figma-to-component`, `prd-round-trip`.

This is the team-library thesis. What you save tonight, your teammate runs next week — without writing a prompt.



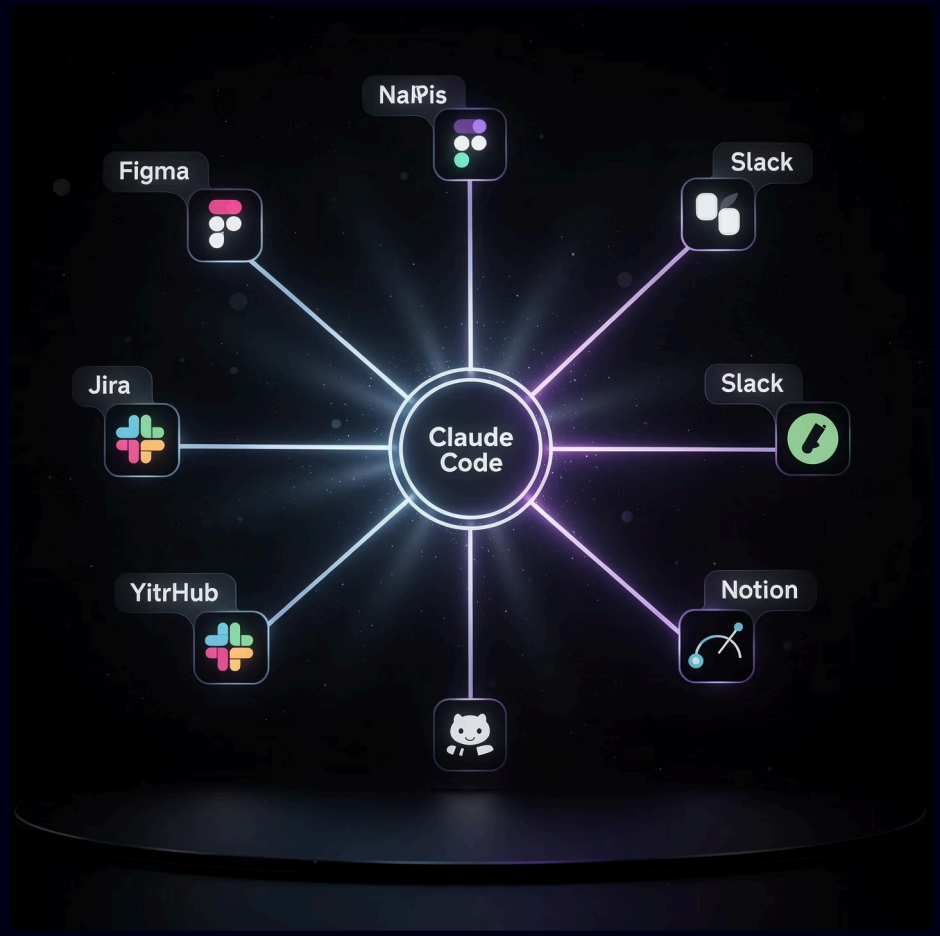
THE FUNDAMENTALS · 4 OF 4

🔌 MCP

Model Context Protocol. How Claude Code reaches outside.

- Connects Claude to external tools and services — Figma, Jira, Slack, GitHub, your own services.
- Pre-configured for tonight in `.mcp.json` — you don't set up anything.
- Claude treats external sources like just another file: reads, writes, acts.

You'll use MCP in Part 3. Claude pulls a Figma frame as if it's reading a local file. Same protocol works for Jira tickets, Slack messages, Notion pages.



POD RULES

Five rules for the next 3.5 hours.

1. **Rotate the keyboard**

   Different person types each Part. Non-negotiable.

2. **Beginner reads aloud**

   This is how the room catches Claude when it's wrong. Say it out loud.

3. **The experienced person is the Challenger**

   Drive the pushback, not the keyboard. Push back when output looks "good enough."

4. **Push back with the prompts on the cheat sheet**

   "Show me where this is true." "What assumptions are you making?"

5. **Ask your pod before you ask me**

   Pedagogy happens at the table. Stuck 3+ min? Then raise your hand.

Today you become the PM/designer your engineers wish they had.

PART 1

Explore.

Onboard to a new product. Audit it. Find the quick wins.

50 minutes · Pod work · One skill saved

THE GROUNDED AUDIT PATTERN

Four steps to audit a codebase you've never seen.

1. SURVEY (plan mode)

Read-only. Map the product. Identify the moving parts. **Don't write the audit yet.**

2. PUSH BACK ON YOURSELF

What did you miss? What's the riskiest thing? What would you tell yourself on day 1?

3. AUDIT

What it is · Features · Three half-built things · Three quick wins · Questions for the CEO.

4. SELF-CRITIQUE

Read again. Find 3 things in your own audit that are wrong, missing, or under-specified.

You just joined this team. The CEO wants a 1:1 in 30 minutes. This is what you bring.

🎬 LIVE DEMO · 14 MINUTES

Watch me do it on InvoiceFlow.

I'll open Claude Code → Local → InvoiceFlow folder. Then run the 4-step pattern out loud. **Watch the prompts. You'll use these.**

01

SURVEY (plan mode ON)

"Survey this codebase. Identify what this product is, every user-facing feature, and the major moving parts. Don't write the audit yet — just the survey."

02

PUSH BACK

"Now push back on what you just said. What did you miss? What's the riskiest part? What did you over-trust from the README?"

03

AUDIT (plan mode OFF)

"Now write the 1-page audit. Format: what this product is · features with file paths · 3 things half-built · 3 quick wins · 3 questions for the CEO. Cite files for every claim."

04

SELF-CRITIQUE

"Read the codebase one more time. Find 3 things in your own audit that are wrong, missing, or under-specified."

⚠ ⚠️ **Watch for:** the moment Claude misses a hotspot. Push back with: *"Look more carefully at /lib/auth.ts — what's actually going on there?"*

✅ **At the end, save as a skill:** *"Save this 4-step pattern as .claude/skills/product-audit.md with proper frontmatter. Make it general enough to work on any codebase."*

🛠️ YOUR TURN — PART 1 · 24 MIN

Before you start.

Type this prompt to begin:

"Use the product-audit skill on this codebase."

You're done when you have:

- A 1-page audit covering: what InvoiceFlow is · all user-facing features with file paths · 3 things half-built (with evidence) · 3 quick wins (with files) · 3 questions for the CEO
- The 4-step pattern saved as a skill in .claude/skills/product-audit.md
- A "I will use this skill when ___" trigger written down

Push back when you hear "good enough":

- *"Show me where in the codebase that's true."*
- *"What assumptions are you making about this?"*
- *"What did you miss?"*

When to stop:

At the **18-minute mark.** Reserve the last 6 min for save-as-skill + writing your trigger.

⏱️ 24 minutes · Rotate the keyboard · Beginner reads aloud

⏳ STAYS UP — 24 MIN

InvoiceFlow Task

ⓘ 📋 You just joined the **InvoiceFlow** team. The CEO wants a 1:1 in 30 minutes. You've never seen the product before. Use Claude Code to produce a 1-page audit you can bring to that meeting.

Your audit needs:

1

What this product is

1 paragraph.

2

Every user-facing feature

1 sentence each + the file or folder.

3

Three things half-built or buggy

With evidence.

4

Three quick-win opportunities

Shippable in a sprint — with files.

SURVEY

PUSH BACK

AUDIT

SELF-CRITIQUE

⏱️ **24 minutes** · Rotate the keyboard · Beginner reads aloud · Then save as .claude/skills/product-audit.md

Part 1 Share-back

Two pods share. Everyone saves. Everyone writes one trigger.

Before you share — write this:

"I will use this skill when ___."

Could be "when I join a new team." Could be "when I inherit a stuck project." One concrete trigger. **Write it now.**

Each sharing pod covers:

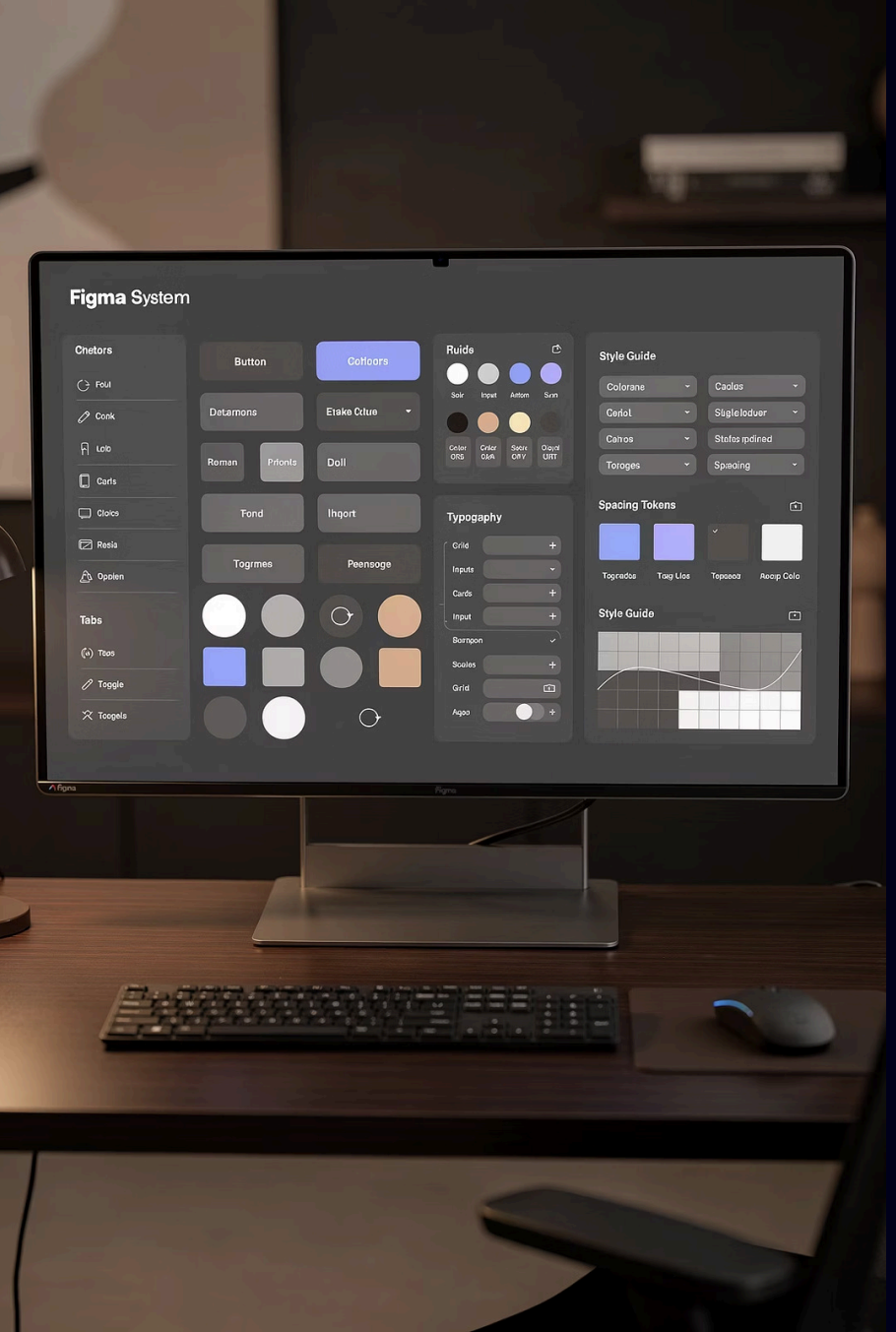
- Their audit highlights — 3 quick wins, 3 CEO questions.
- The skill prompt they saved.
- Their "I will use this skill when ___" trigger.

✅ **Skill saved?** Hold up your laptop showing .claude/skills/product-audit.md

☕ Break

Stretch. Refill. Don't open Slack.

10 minutes — set a phone timer.



PART 2

Constrain.

Figma + Claude + your design system. Generate code that respects the system — not freelance pixels.

50 minutes · Pod work · One skill saved

THE SYSTEM-FIRST PATTERN

Four steps to generate components that respect your system.

1. READ THE SYSTEM FIRST

tokens.json, components/ui/, CLAUDE.md. **Output what's available before touching the Figma file.**

2. READ THE FIGMA FRAME

Map every visual element to a token or primitive. **No match? Don't invent — add to "Questions for designer."**

3. GENERATE

Use only system primitives. Mark every deviation with // SYSTEM-BEND: for full visibility.

4. SELF-REVIEW

Where the system fit. Where you bent. What goes back to the designer. **This list is the artifact.**

⚠ ⚠️ **Demo begins with the WRONG approach** — generating without reading the system first. Watch Claude invent tokens. That failure anchors everything that follows.

🎬 LIVE DEMO · 15 MINUTES

Watch me do it wrong, then right.

❌ The WRONG way (3 min)

"Generate the Invoice component from this Figma frame: https://www.figma.com/design/Lmdeg4ItRaPaspVcH0OP7A/..."

⚠️ **Watch:** Claude invents colors. Uses raw hex. Names primitives that don't exist. This is the failure mode you need to recognize.

✅ The RIGHT way: System-first (10 min)

1. **READ SYSTEM:** *"Read the design system: tokens.json, components/ui/, CLAUDE.md. List what's available — tokens, primitives, conventions."*
2. **READ FIGMA:** *"Now read the Figma frame at [URL]. List every element. Map each to a system token or primitive. If no match, add to a 'Questions for designer' list."*
3. **GENERATE:** *"Now generate the component. Use ONLY the primitives and tokens you identified. No raw hex. Mark every deviation with // SYSTEM-BEND:."*
4. **SELF-REVIEW:** *"Review your component. Find 3 bends that weren't necessary. Be honest."*

✅ **Save as a skill:** *"Save this 4-step pattern as .claude/skills/figma-to-component.md."*

🛠️ YOUR TURN — PART 2 · 23 MIN

Before you start.

Type this prompt to begin:

"Use the figma-to-component skill to generate the InvoiceLineItem component from [Figma URL]."

You're done when you have:

- A component file in components/ that uses ONLY system primitives + tokens (zero raw hex)
- Every deviation marked with // SYSTEM-BEND: and a reason
- A "Questions for designer" list (this is the actual prize of this Part)
- The skill saved as .claude/skills/figma-to-component.md

Push back when Claude bends the system:

- *"Show me where in tokens.json that value exists."*
- *"Use the existing primitive instead — which one is closest?"*
- *"Add this to the 'Questions for designer' list — don't invent."*

When to stop:

At the **18-minute mark.** Reserve the last 5 min for save-as-skill + trigger writing.

⏱️ 23 minutes · Rotate the keyboard · **Designer reads aloud** (rule swap for this Part)

⏳ STAYS UP — 23 MIN

Figma → Component Task.

ⓘ 📋 Your designer just shipped a Figma file with a new Invoice component — a complete invoice template with header details, a line-item table, and a footer. Your design system lives in /components/ui with tokens in tokens.json. Use Claude Code with the Figma MCP to generate the component — and make sure it uses **the system, not custom values.** If it doesn't, push back on Claude until it does.

The key discipline:

> If Claude invents a color or spacing value, stop it. Ask: *"Show me where in tokens.json that value exists."* **If it doesn't exist, it doesn't ship.**

READ SYSTEM

READ FIGMA

GENERATE

SELF-REVIEW

⏱️ **23 minutes** · Rotate the keyboard · Designer reads aloud · Then save as .claude/skills/figma-to-component.md

Share-back

One pod shares. Everyone saves. Everyone writes one trigger.

Before you share — write this:

"I will use this skill when ___."

Could be "when a designer hands off a component." Could be "when I'm reviewing a PR that touches the UI."

The sharing pod shows:

- Their generated component — open the file in Claude Desktop's file viewer.
- Their **"Questions for designer"** list — the most useful artifact of this Part.
- Their *"I will use this skill when ___"* trigger.

✅ **Skill saved?** .claude/skills/figma-to-component.md — the "Questions for designer" list is the prize. It surfaces system gaps.

☕ Break

Two more parts to go. You're past halfway.

10 minutes.

PART 3

Collaborate.

Customer feedback → PRD → implementation → revised PRD. PM ↔️ Dev ↔️ PM, all in one session.

50 minutes · Pod work · One skill saved

THE ROUND-TRIP PATTERN

Synthesize. Three roles. Same Claude.

0. SYNTHESIZE

Scan feedback.csv. Find the top theme. Define the feature. Let real user voice drive — **don't skip this step.**

1. PM HAT (Role 1)

Write the PRD grounded in real code. Save to docs/prd-{feature}.md. Use the skeleton — don't start cold.

2. DEV HAT (Role 2)

Switch hats. Read the PRD cold. Implement. **Note every ambiguity** — that list is half the lesson.

3. PM HAT AGAIN (Role 3)

Read the diff. Revise the spec. Save to docs/prd-{feature}-v2.md. **This is what you really learn.**

What you really learn: how your spec process actually works — and where it breaks down.

🎬 LIVE DEMO · 17 MINUTES

Watch me run the round-trip on real customer feedback.

0. SYNTHESIZE

"Read workshop-data/feedback.csv (200 rows). Find the top theme by impact and frequency. Define a concrete feature that addresses it — files involved, acceptance criteria. No vague 'improve UX'."

1. PM HAT (Role 1)

"PM hat on. Open docs/prd-template.md — the 3-line skeleton. Fill it in for the feature you just defined. Use the codebase as evidence. Cite specific files."

2. DEV HAT (Role 2)

"Now switch to Dev hat. Implement the feature. As you go, note every ambiguity in the PRD that's slowing you down. Quote the PRD line and explain why."

3. PM HAT AGAIN (Role 3)

"Read the diff you just made. Read the PRD again. Revise it to fix every ambiguity. Be honest — if you cut corners as Dev, say so as PM."

✅ **Save as a skill:** *"Save this round-trip pattern as .claude/skills/prd-round-trip.md. Include the synthesize step at the top."*

⚠️ **The honest moment:** if I run out of time, I'll stop after Role 2 and tell the room why. Skipping Role 3 honestly is the lesson.

🛠️ YOUR TURN — PART 3 · 23 MIN

Before you start.

Type this prompt to begin:

"Use the prd-round-trip skill on workshop-data/feedback.csv. Pick a top theme and run the full 0/1/2/3 cycle."

You're done when you have:

- A real PRD in docs/prd-{feature}.md with file evidence
- Real code changes (run git diff to see them)
- A revised PRD in docs/prd-{feature}-v2.md with the ambiguities fixed
- The skill saved as .claude/skills/prd-round-trip.md
- Your "I will use this skill when ___" trigger

Push back when Claude is vague:

- *"That's too abstract. Give me a specific feature an engineer could code tomorrow."*
- *"Quote the PRD line that was ambiguous."*
- *"What did you cut as Dev that PM-you should know about?"*

If running out of time:

⚠ **Stop after Role 2 (Dev hat).** Skipping Role 3 honestly is more useful than faking it. The "we ran out of time" report is the most honest deliverable.

⏱️ 23 minutes · Whoever didn't type in Parts 2–3 types now

⏳ STAYS UP — 23 MIN

PRD Round-Trip Task.

ⓘ 📋 200 rows of real customer feedback wait at workshop-data/feedback.csv (English + Hebrew, mixed sentiment, with theme tags). A 3-line PRD skeleton waits at docs/prd-template.md — **fill it in, don't start from scratch.**

Run the full round-trip in 23 minutes:

1

0. Synthesize

Find the top theme. Define the feature. Don't skip this.

2

1. PM hat (Role 1)

Fill in the PRD skeleton against the codebase.

3

2. Dev hat (Role 2)

Implement. Note every ambiguity.

4

3. PM hat again (Role 3)

Read the diff. Revise the PRD.

⚠ ⚠️ **If running out of time: stop after Role 2.** Better to skip Role 3 honestly than fake it. The honest "we ran out of time" report is the most useful one.

⏱️ **23 minutes** · Whoever didn't type in Parts 2–3 types now · Then save as .claude/skills/prd-round-trip.md

PRD Round-Trip Share-back

Three pods share what they shipped.

Before you share — write this:

"I will use this skill when ___."

This is the **most important trigger of the day.** Be specific. *"When the next feature debate happens at standup"* is fine. *"Eventually"* isn't.

Each pod, 90 seconds:

- The feature they chose.
- One thing the spec got wrong.
- One thing they learned about their own spec process.
- Their "I will use this skill when ___" trigger.

✅ **Skill saved?** .claude/skills/prd-round-trip.md — that's three entries in your team library. **The loop closes here.**

Where the line is moving.

FOUR TRAJECTORIES

Four trajectories that matter for product people.

2026	2027
One agent on your desktop. Plan mode, slash commands, MCP.	Multi-agent teams. PM + Dev + QA agents — by default.
Autonomous for 30 minutes. Check in, redirect, approve.	Autonomous for hours. Background work while you sleep.
Agents in the IDE. Agents in CI and the org.	Part of release. Owning a Slack channel.
Skills as personal scratch files.	Skills as the team operating system. Shared library. Forkable. **The PMs who started theirs in 2026 are years ahead.**

The patterns you saved tonight — plan mode, CLAUDE.md, skills, MCP — are concepts the whole industry is converging on. **They transfer to whatever tool wins in 2027.**

What it means for your role.

The PM/designer with these tools is not replacing engineers.

Old role	New role
Wrote a PRD. Handed it off. Waited.	Writes a PRD grounded in real files.
Asked "is this feasible?" → waited 2 days.	Answers "is this feasible?" in 5 minutes — with evidence.
Watched engineers build something different.	Catches the drift before it ships.
Filed Jira tickets to fix the drift.	Skips the Jira loop.

The skill is **steering, scoping, judgment** — at higher leverage. Remember the three words from tonight: **Explore, Constrain, Collaborate.** Monday, you'll do one of them on something at work. Engineers will notice in week one. Your VP will notice in month one.

HONEST BOUNDARIES

Three things Claude Code is NOT for.

❌ Quick conversational thinking

→ Use Claude.ai in the browser. Faster. Less overhead. No file access needed.

❌ Disposable prototypes

→ Use Lovable, Bolt, or v0. Better for marketing pages and throwaway demos.

❌ Anything you wouldn't show your engineering manager

→ The audit trail is real. Claude edits real files. Treat every session like work that ships.

The strongest skill is knowing when not to reach for the tool. That's judgment — and it's the same skill that makes the rest of tonight land.

Write your line. Take it home.

> **By Friday I will ship one [PRD / component / workflow] using Claude Code.**
>
> *Sticky note. Laptop sleeve. Not Notion.*

Take a sticky note. Write your version of this line. Pick one of the three skills. Pick a real feature. Pick a real Friday. Put it in your **physical** laptop sleeve — where you'll see it on Monday.

📝 I'll check the Slack channel one week from today. The honest reports — including *"I tried and it didn't work"* — are the ones I want to read most.

Resources

Four things to take with you.

1

Slack/Discord channel

Where the cohort lives. Share your skills here. Fork others'. Build the library. **Use it before Google. Use it before ChatGPT.** The cohort knows your context.

2

Sample repo

braightwave/claude-code-pm-starter — clone it, fork it, make it yours.

3

Cheat sheet

The 1-pager on your table. PDF link in the channel. 6-prompt pushback playbook included.

4

Graduate contacts

The four panelists are reachable in the channel. Real people who've shipped this. Use them.

The channel's library starts as 3 entries each tonight. By month's end it's 100. By Q3 it's the BrAIght Wave PM Library. You're contributors, not consumers.

You came in today as PMs and designers.

You're leaving as the PM and designer your engineers wish they had.

See you in the channel.